AICE2005 - Systematic Design

Formal Specification and Verification

Asieh Salehi Fathabadi
University of Southampton
2026-2027

Week 4-5 Overview

- Lecture 1: Introduction to Formal Methods
- Lecture 2: Formal Specification Techniques
- Lecture 3: Mathematical Foundations
- Lecture 4: Formal Verification Methods
- Lecture 5: Tools and Applications

LECTURE 1:

INTRODUCTION TO FORMAL METHODS

Lecture 1 Overview

- What are **Formal Methods**?
- Historical Context & Motivation
- Types of Formal Methods
- Industry Applications

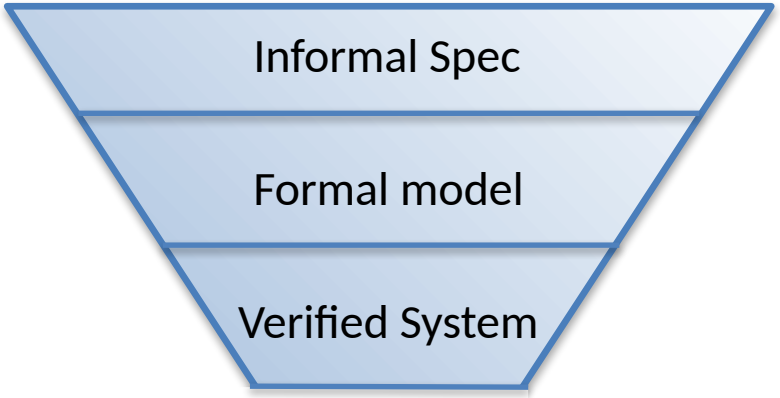
Learning Objectives:

- Define formal methods and their role in systems engineering
- Identify when formal methods are appropriate to use
- Classify different types of formal methods

Why Formal Methods?

Formal Methods = Design it right, not just test it later.
turn uncertainty into proof

- **Software is everywhere:** from AI to aerospace, errors can be *catastrophic*
- **Traditional testing ≠ enough:** can only show *presence*, not *absence* of bugs
  - Ambiguous requirements → Precise, mathematical specifications
  - Hidden design errors → Early detection via proofs & formal analysis
  - Complex AI systems → Rigorous reasoning about safety & security



Formal methods = mathematical confidence in correctness

What are Formal Methods?

Formal methods are mathematically rigorous techniques for the specification, development, and verification of software and hardware systems.

Key Characteristics:

- **Mathematical Foundation**: Based on mathematical logic, set theory, and discrete mathematics
- **Precision**: Eliminate ambiguity through precise mathematical notation
- **Rigor**: Enable mathematical reasoning about system properties
- **Tool Support**: Automated analysis and verification capabilities

Informal vs Formal Methods:

- **Informal**: Natural language requirements → Ambiguous interpretations → Manual review and testing → Error detection during testing
- ***Formal***: Mathematical specifications → Precise, unambiguous meaning → Mathematical proof and verification → Error prevention during design

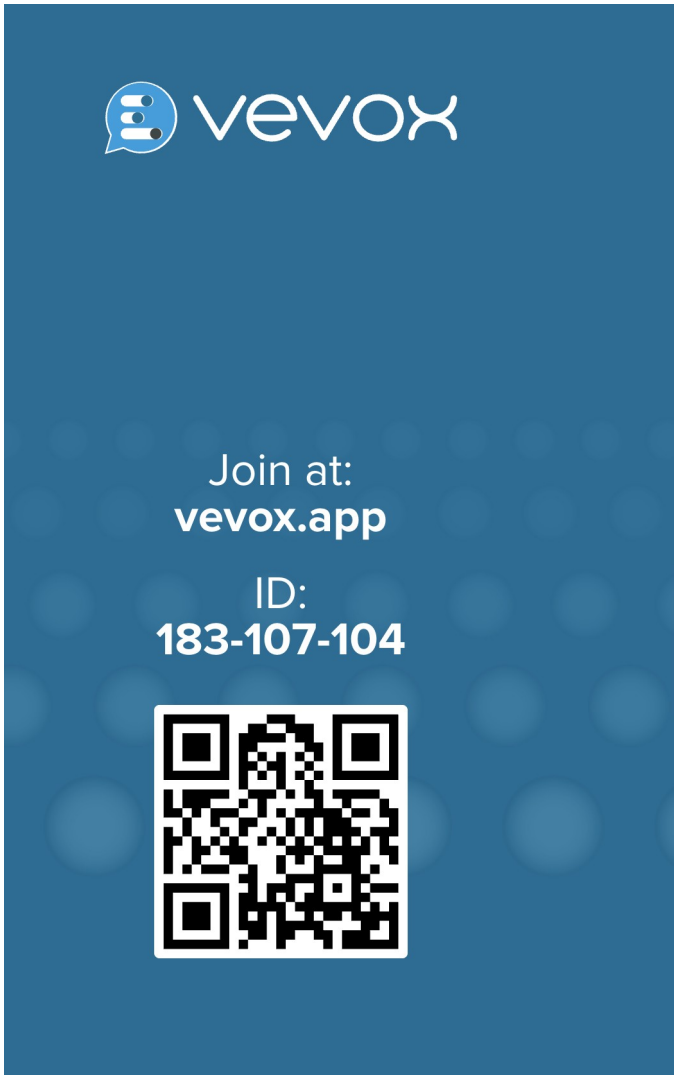
Formal Modelling vs Programming

Formal modelling helps ensure the **design is correct** before writing any code.
Formal modelling: "Prove it."
Programming: "Make it run."

Formal Modelling	**Programming**
Focuses on **what the system should do**	Focuses on **how to implement it**
Abstract, high-level representation	Concrete, low-level instructions to a computer
Uses mathematical notation (e.g., sets, logic, invariants)	Uses executable programming languages (e.g., Python, C, Java)
Detects design errors early (before coding)	**Errors often discovered later (testing/debugging)**
Prioritizes correctness, consistency, and safety	Prioritizes functionality and performance
Tool support: proof, model-checking, animation	Tool support: compilers, debuggers, test frameworks

Let's think and reflect

Informal - Semi-Formal - Formal



Historical Context & Famous Failures

Software Crisis of 1960s-70s:

- Problem: Projects consistently over budget, late, unreliable
- NATO Conference (1968): Coined "software engineering"
- Response: Need for **more rigorous** development methods

Costly Software Failures:

- 1994 - Intel Pentium: Floating-point division bug ($475 million recall)
- 1996 - Ariane 5: Integer overflow in guidance ($370 million rocket destroyed)
- 1999 - Mars Climate Orbiter: Unit conversion error ($125 million spacecraft lost)
- 2003 - Northeast Blackout: Race condition in alarm system (55 million people affected)

Types of Formal Methods

Specification Methods:

- **Z Notation**: Set theory, predicate logic → Data-intensive systems → Z/EVES, CZT
- **B Method**: Set theory, refinement → Sequential systems → Atelier B, ProB
- **Event-B**: Set theory, events → Distributed/concurrent systems → Rodin Platform
- **VDM**: Model-based specification → Sequential systems → VDMTools, Overture
- **CSP**: Process algebra → Concurrent systems → FDR, PAT
- **TLA+**: Temporal logic, set theory → Distributed systems → TLC, TLAPS

Verification Methods:

- **Model Checking:** Automatically explores all possible system states to ensure no bad behavior can occur → Exhaustive state space exploration
- **Theorem Proving:** Interactive proof construction: uses mathematical logic and proofs (often with human guidance, Interactive) to show a system always meets its specification.
- **Static Analysis:** Analyzes source code without running it to find potential bugs and vulnerabilities.
- **Runtime Verification:** Monitors a running system to ensure its behavior matches expected properties during execution.

When to Use Formal Methods

Strong Candidates:

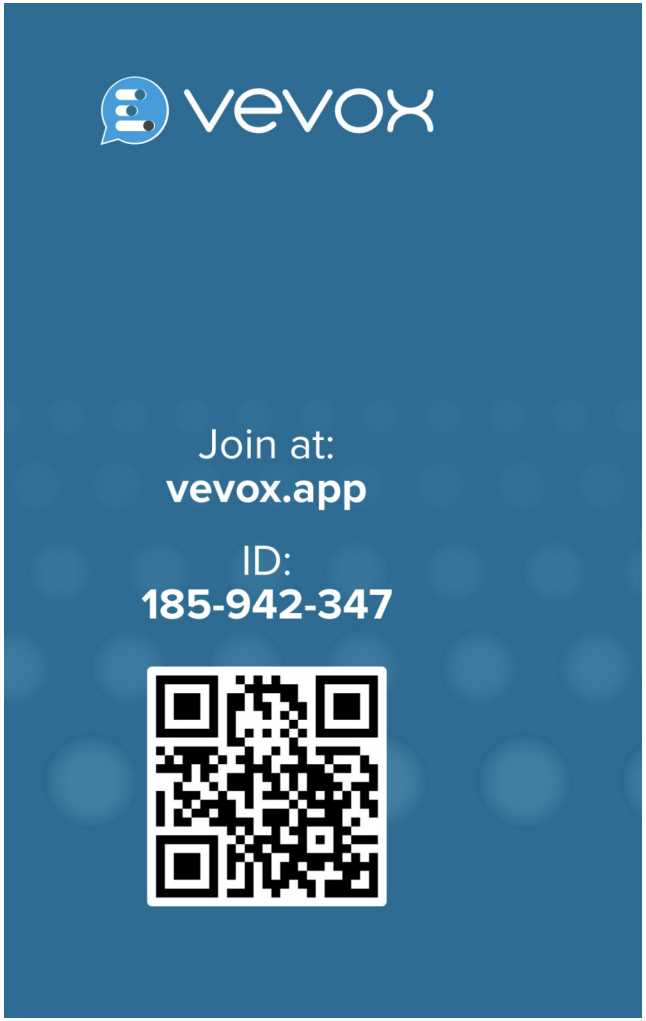
- **Safety-critical systems**: Lives depend on correctness
- **Security-critical systems**: High-value assets to protect
- **High-cost failures**: Financial impact of bugs is severe
- Regulatory requirements: Standards mandate formal methods
- Complex concurrent systems: Race conditions possible

Cost-Benefit Analysis:

- **Learning curve**: High initial training effort → pays off through fewer defects and rework
- **Tooling Costs:** Licensing investment → typically positive ROI in high-assurance domains
- **Specification Effort:** Requires upfront rigor → yields clearer requirements and fewer design errors
- **Maintenance:** Moderate ongoing effort → improved system maintainability

Let's think and reflect

When to use Formal Methods (FM)



Industry Applications

Aerospace & Defense:

- Flight Control Systems: Airbus A380 uses SCADE
- Avionics Software: DO-333 recommends formal methods for Level A
- Satellite Systems: NASA uses formal verification
- Military Systems: Cryptographic protocols

Transportation:

- Paris Metro Line 14: Event-B for automatic train control
- Railway Signaling: B Method for interlocking systems
- Automotive: ISO 26262 promotes formal methods

Medical Devices:

- Pacemakers: Boston Scientific uses formal verification
- Insulin Pumps: FDA encourages formal methods
- Radiation Therapy: Post-Therac-25 requirements

Success Story - Paris Metro Line 14:
Event-B for Automatic Train Operation system
Zero safety incidents since 1998

Summery

- **What is a Formal Method (FM):**
  mathematical techniques to specify and verify sw and hw systems
- **Why FM:**
  earlier bug detection
- **When FM:**
  Safety-critical and Security-critical Systems
- **Types of FM:**
  Z, B, Event-B, CPS, TLA, …

Let's read and explore

- Systems Engineering Principles and Practice, Alexander Kossiakoff et. al., 2011
- Modeling in Event-B: System and Software Engineering
  by Jean-Raymond Abrial, 2010